

Domain 1 — Section Review



Agentic Architecture & Orchestration

10/11

91% · Strong · 1 missed

Q1 ✓ CORRECT

Production data shows that in 12% of cases, the support agent skips `get_customer` and calls `process_refund` directly using only the customer's stated name, occasionally leading to misidentified accounts and incorrect refunds. What change most reliably fixes this?

- A Strengthen the system prompt: "Customer verification via `get_customer` is mandatory before any order operation."
- B Add few-shot examples always showing `get_customer` called first, even when the customer volunteers an order ID.
- C Add a programmatic prerequisite that blocks `lookup_order` and `process_refund` until `get_customer` has returned a verified customer ID.**
- D Deploy a routing classifier that analyses each request and enables only the appropriate tool subset.

Why C is correct:

Deterministic > probabilistic when stakes are real. Prompt instructions and few-shot examples have non-zero failure rates (~95–99%). When a wrong sequence causes financial harm, only programmatic enforcement (hooks or prerequisite gates) guarantees the ordering 100% of the time. Option D addresses tool availability, not tool ordering.

Q2 ✓ CORRECT



After running the system on the topic "impact of AI on creative industries," you observe that each subagent completes successfully: the web search agent finds relevant articles, the document analysis agent summarises papers correctly, and the synthesis agent produces coherent output. However, the final reports cover only visual arts, completely missing music, writing, and film production. When you examine the coordinator's logs, you see it decomposed the topic into three subtasks: "AI in digital art creation," "AI in graphic design," and "AI in photography." What is the most likely root cause?

- A The synthesis agent lacks instructions for identifying coverage gaps in the findings it receives from other agents.
- B The coordinator agent's task decomposition is too narrow, resulting in subagent assignments that don't cover all relevant domains of the topic.**
- C The web search agent's queries are not comprehensive enough and need to be expanded to cover more creative industry sectors.
- D The document analysis agent is filtering out sources related to non-visual creative industries due to overly restrictive relevance criteria.

Why B is correct:

Trace failures to their origin. The coordinator's logs reveal the root cause directly — it decomposed 'creative industries' into only visual arts subtasks. The subagents executed their assigned tasks correctly; the problem is what they were assigned. Options A, C, and D incorrectly blame downstream agents that are working correctly within their (too-narrow) brief. Missing topics = coordinator decomposition problem.

Q3 ✓ CORRECT

When researching a broad topic, you observe that the web search agent and document analysis agent are both investigating the same subtopics, resulting in significant overlap in their findings.

Token usage has nearly doubled without proportionally increasing the breadth or depth of research coverage. What is the most effective way to address this?



- A** Give both agents identical tool sets and let the model naturally differentiate their outputs based on tool capabilities.
- B** Merge the web search and document analysis agents into a single research agent to eliminate the coordination overhead.
- C** Partition research scope in the coordinator's decomposition — assign distinct subtopics or source types to each agent to minimise duplication.
- D** Add a deduplication pass where the synthesis agent filters out redundant findings after both agents complete.

Why C is correct:

Duplicate work = coordinator partitioning problem. The coordinator is responsible for partitioning research scope to minimise overlap — assigning distinct subtopics or source types (e.g., web search covers academic/news, document analysis covers uploaded PDFs). Merging agents (B) sacrifices useful specialisation. Deduplication (D) wastes tokens on work that was already redundant — fix upstream, not downstream.

Q4 ✓ **CORRECT**

A colleague suggests having the document analysis agent send its output directly to the synthesis agent instead of routing through the coordinator. What is the main advantage of keeping the coordinator as the central hub for all subagent communication?

- A** It reduces coordinator involvement in routine data passing, decreasing overall latency.

- B** The coordinator provides centralised observability, consistent error handling, controlled information flow across the entire pipeline.



- C** Direct communication allows subagents to share intermediate context the coordinator wouldn't otherwise see.

- D** It prevents the synthesis agent from having to wait for the coordinator to reformat results before processing them.

Why B is correct:

Hub-and-spoke: all communication routes through the coordinator. Routing everything through the coordinator enables: (1) centralised observability — every message is logged and traceable, (2) consistent error handling — the coordinator decides recovery strategy, (3) controlled information flow — the coordinator decides what context each subagent receives. Direct subagent-to-subagent communication bypasses all three.

Q5 ✗ INCORRECT

Your agent runs retries on transient API failures, but customers occasionally report being charged twice after a network timeout. Investigation shows the server processed the first request before the timeout. The retry then issued a second charge. Which fix addresses this completely?

- A** Fix the backoff policy — longer delays between retries reduce duplicate processing on the server.

- B** Add idempotency keys only — a unique key per operation prevents the server from processing the same request twice.

- C** Fix the retry policy to classify errors before retrying — transient failures retry, permanent failures don't.

D Both error classification AND idempotency keys are required — either alone i



ff



Why D is correct:

The paired-fix pattern. Error classification alone won't prevent duplicates if the server already processed a request before timing out. Idempotency keys alone won't help if a request that never reached the server is never retried. You need both: classify errors correctly so only safe retries happen, AND use idempotency keys so safe retries don't double-process. Generate a UUID/hash of {operation + customer + amount} on the first attempt.

Q6 ✓ **CORRECT**

To execute the web search subagent and document analysis subagent in parallel rather than sequentially, the coordinator must:

A Use `fork_session` for each parallel branch before delegating.

B Emit multiple Task tool calls across separate coordinator turns, one per subagent.

C Emit multiple Task tool calls within a single coordinator response.

D Set `allowedTools` to list all subagent type names simultaneously.

Why C is correct:

Parallel = multiple Task calls in one response. Emitting Task calls across separate turns means sequential execution — each turn waits for the prior. Parallel execution only happens when multiple Task tool calls appear in a single response. `fork_session` creates independent branches from a shared analysis baseline — useful for exploring divergent approaches, not for parallelising different agent types.

Q7 ✓ CORRECT



The document analysis subagent frequently encounters failures when processing PDFs. Some have corrupted sections causing parsing exceptions, others are password-protected, and occasionally the parsing library times out on large files. Currently, any exception immediately terminates the subagent and returns an error to the coordinator, which must decide whether to retry, skip, or fail the entire task. This is causing excessive coordinator involvement in routine error handling. What's the most effective architectural improvement?

- A Add a coordinator-level retry policy that re-invokes the subagent up to 3 times for any document that fails.
- B Add detailed error-handling instructions to the subagent's system prompt so it classifies and responds to each failure type.
- C Pre-validate all PDF files before sending them to the subagent, rejecting password-protected or oversized files upfront.
- D Implement local error recovery within the subagent: try fallback parsers for corrupt sections, skip unrecoverable sections with a partial-result flag, and return structured error context only for truly unresolvable failures.

Why D is correct:

Local recovery within subagents — propagate only what can't be resolved locally. Transient and recoverable failures (corrupted sections, library timeouts) should be handled inside the subagent with fallbacks, partial results, and a flag indicating what was skipped. The coordinator should receive structured error context only for truly unresolvable failures. This eliminates excessive coordinator involvement in routine handling while preserving recovery information for complex cases. Coordinator-level retry (A) invokes the full subagent again on every failure — expensive and unnecessary for recoverable issues.

Q8 ✓ CORRECT

You've defined a coordinator agent with a system prompt instructing it to delegate research subtasks to specialized subagents. During testing, the coordinator produces detailed responses but never delegates to any subagent — it handles everything itself. The most likely cause is:

- A The coordinator's system prompt doesn't explicitly name which subagents to invoke.
- B The subagent definitions don't have system prompts configured.
- C `"Task"` is not included in the coordinator's `allowedTools` configuration.
- D The coordinator needs more context about the research topic before it will decide to delegate.

Why C is correct:

allowedTools must include "Task" for a coordinator to spawn subagents. The Task tool is the mechanism for subagent invocation. Without it in allowedTools, the coordinator literally cannot call subagents regardless of what its system prompt says. This is a configuration requirement, not a prompting problem. Subagent system prompts (B) don't affect whether the coordinator can call them.

Q9 ✓ **CORRECT**

Your automated code review runs three steps on every PR: (1) analyze each file individually for local issues, (2) run a cross-file integration pass for data flow and contract issues, (3) generate a severity-ranked summary. The structure never varies. Which decomposition pattern is this, and is it the right choice?

- A Dynamic decomposition — appropriate because findings from step 1 could change what step 2 needs to examine.

B Fixed sequential pipeline (prompt chaining) — appropriate because the structure is defined and each step consumes the prior step's output.



C Evaluator-optimiser — appropriate because step 3 evaluates the quality of the prior steps.

D Fixed sequential pipeline — but wrong; dynamic decomposition would skip unnecessary passes on simple PRs.

Why B is correct:

Fixed sequential pipeline for stable, predictable workflows. Prompt chaining is the right pattern when the structure doesn't need to adapt based on intermediate findings — each step consumes the prior step's output in a defined sequence. Dynamic decomposition adds overhead without benefit here since all three passes are always valuable regardless of PR content. Evaluator-optimiser (C) requires an objective quality signal to iterate against.

Q10 ✓ CORRECT

You want to explore two competing architectural approaches for the research pipeline — a sequential pipeline vs a parallel fan-out design — starting from the same codebase analysis you completed this morning. You need both explorations to stay independent so findings don't cross-contaminate. Which mechanism is designed for this?

A Open two separate Claude Code terminal sessions and manually synchronize findings at the end.

B Use `--resume` with two different session names, one per approach.

C Use `fork_session` to create two independent branches from the shared analysis baseline.

D Run two Task tool calls in a single coordinator response, one per approach.

**Why C is correct:**

fork_session creates independent exploration branches from a shared analysis baseline. Both branches start with the same codebase understanding; their findings don't contaminate each other. --resume (B) continues a session linearly — it doesn't branch. Two terminal sessions (A) don't share the prior analysis. Task tool calls (D) are for spawning subagents in a multi-agent system, not for branching your own exploration.

Q11 ✓ **CORRECT**

Last week you ran a deep codebase exploration session, mapping authentication flows and service boundaries. Since then, the team merged a major refactor of the auth module. You want to continue analysis using the prior findings. What's the most reliable approach?

- A** Resume the prior session using `--resume` — it already has the full codebase map and tool results.
- B** Start a new session and re-explore the entire codebase from scratch to avoid any stale assumptions.
- C** Start a new session, inject a structured summary of the valid prior findings, and explicitly note that the auth module has changed.
- D** Resume the prior session and instruct Claude to ignore its previous observations about the auth module.

Why C is correct:

New session + structured summary when prior tool results are stale. The auth module refactor means prior tool results for that area are stale — resuming imports them as if current. The reliable pattern: start fresh, inject a summary of findings that remain valid, and call out what changed so the new session investigates correctly. Resume is appropriate when prior context is mostly valid — here, a major

refactor invalidates a key portion.



Next Section →